

Trustworthy Credit Risk Scoring: Calibrated, Explained and Fair Loan Decisions with a Guarantee

Amartya Gawali (EP23BT010)

PRML course project, IIT Dharwad, 2026

Code: <https://github.com/INKiE1911/trustworthy-credit-risk>

Live demo: <https://trustworthy-credit-risk-richtsc9esc5kj5mvuq86kb.streamlit.app/>

Abstract—A lender has to decide, for every applicant, whether to lend. We build a complete loan-default system on the Home Credit Default Risk data (307,511 labelled applications in 7 linked tables) and ask not only how accurate it is but whether each of its outputs can be trusted. A tuned LightGBM on 240 leak-free features reaches a test ROC-AUC of 0.790 (95% CI 0.784–0.797), exactly its cross-validation estimate, and ranks applicants significantly better than logistic regression, a glass-box EBM and a bank-style points scorecard. Its probabilities are already calibrated (expected calibration error 0.0018), so a break-even money rule earns 13.9% more than approving everyone. A class-conditional conformal layer approves, refers or declines with a written promise: on the untouched test split only 9.8% of defaulters were auto-approved, against a promised 10%. Every decision is explained with TreeSHAP reason codes and an actionable “what would change it” answer; fairness is audited by gender and age (reweighing cuts the gender gap in wrongly declined good customers from 4.6 to 2.0 points at equal accuracy); and drift is watched with the population stability index. The report explains each idea from first principles, with worked examples on the real numbers.

I. INTRODUCTION

A. The problem

When someone applies for a loan, the lender must answer one question: *will this person repay?* Two kinds of mistakes are possible:

- approving someone who then *defaults* (does not repay): the lender loses money;
- declining someone who *would* have repaid: the lender loses a good customer, and the person loses access to credit.

The two mistakes do not cost the same, and if the second one happens more often to some groups of people than to others it can be unfair and, in many countries, unlawful.

Machine learning can learn from past loans which applicants tend to default. This is a *supervised classification* problem: every past application has a label (defaulted or not), and the model learns a function from the application’s data to a *probability of default*, a number between 0 and 1.

B. What “trustworthy” means here

A model that is merely accurate is not enough for a real lending decision. We break trust into six questions, and each later section answers one of them:

- 1) **Does it rank applicants well?** Are risky people scored higher than safe ones (Sec. VI, VII)?
- 2) **Are its probabilities honest?** When it says 10%, do about 10% of such people default (Sec. VIII)?
- 3) **Can its errors be controlled?** Can we promise an upper limit on wrongly approved defaulters and wrongly declined good customers (Sec. VIII)?
- 4) **Can it explain itself?** Can it say *why* an applicant was declined, and what would change the answer (Sec. IX)?
- 5) **Is it fair?** Does it treat men and women, young and old, similarly (Sec. X)?
- 6) **Does it keep working?** Will we notice when new applicants stop looking like the old ones (Sec. XII)?

C. How the project was run

The work was done in 18 numbered steps on the Kaggle Home Credit Default Risk data [1]. Three rules were kept throughout: (i) every number in this report comes from a notebook in the repository; (ii) every model is compared on the same five cross-validation folds, with confidence intervals; (iii) 20% of the data, the *test split*, was locked away and opened exactly once, at the very end, so the final numbers are an honest estimate of performance on new applicants.

Contributions. (i) A leak-free feature pipeline over 7 related tables (240 features) and a ladder of 14 models compared with bootstrap confidence intervals and paired tests. (ii) Two controlled experiments: class-imbalance “fixes” make the model worse, and SMOTE is recognised by the trees; three tuning methods at the same budget. (iii) Seven classical algorithms written from scratch in NumPy that match scikit-learn. (iv) A decision layer: a calibration check, a break-even money threshold, and a conformal approve/refer/decline rule whose error promise was verified on the test split. (v) Explanations (TreeSHAP, a LIME check, PDP, actionable counterfactuals), a fairness audit with three mitigations and an impossibility demonstration on the real base rates, risk personas, and a drift monitor. (vi) A working service (FastAPI + Streamlit, in Docker) with a public demo that uses only made-up applicants and a stand-in model, so neither the real data nor the real model is published.

Table I maps the work to the 16 units of the course.

TABLE I
SYLLABUS COVERAGE: WHERE EACH COURSE UNIT APPEARS IN THE PROJECT.

#	Unit	What this project does
1	Introduction	Credit scoring framed as pattern recognition with unequal error costs (Sec. I, VIII).
2	Types of ML	Supervised (default prediction), unsupervised (segments, personas), semi-supervised as the reject-inference problem (Sec. XIII), reinforcement learning as future work.
3	Preprocessing	Data traps (365243 code, XNA), imputation, one-hot / target / WoE encoding, scaling, 7-table aggregation, feature selection inside each fold.
4	Regression	Logistic regression (Newton's method from scratch); ridge vs least squares to predict EXT_SOURCE_1 ($R^2 = 0.405$).
5	Classification	Decision tree (depth curve), KNN, linear and RBF SVM, Naive Bayes; confusion matrix and all metrics.
6	Clustering	K-means++, Gaussian mixture with EM and BIC, Ward hierarchical, HDBSCAN; silhouette, Davies–Bouldin, Calinski–Harabasz, adjusted Rand index.
7	Dimensionality reduction	PCA and Fisher LDA from scratch; scree plot; t-SNE at three perplexities.
8	Ensembles	Bagging (random forest, out-of-bag score), AdaBoost, gradient boosting (LightGBM, XGBoost, HistGB), EBM.
9	Neural networks	NumPy MLP with hand-written backpropagation (gradient check 2×10^{-7}); PyTorch MLP with embeddings.
10	Probabilistic models	MLE vs MAP (L2 = Gaussian prior), Laplace Bayesian logistic regression, Gaussian NB, EM, calibration.
11	Evaluation	Stratified frozen splits, 5-fold CV, ROC/PR/KS, bootstrap CIs, paired bootstrap, McNemar, 5×2 cv t-test, Holm correction, learning curves.
12	Tuning	Grid vs random vs Bayesian optimisation (Optuna TPE) at the same budget.
13	Practical ML	Python package, 14 notebooks, 133 unit tests, continuous integration, MLflow tracking.
14	Explainable AI	Gain, permutation and SHAP importance, PDP/ICE, LIME, scorecard, EBM, counterfactuals, reason codes.
15	Applications	A complete credit-decision system with an API, a web app and monitoring.
16	Ethics	Fairness audit, proxy test, impossibility demonstration, 3 mitigations, privacy, model card, datasheet, human review.

II. BACKGROUND: THE KEY IDEAS

This section explains, from the beginning, the measuring tools used everywhere else.

A. From a probability to a decision: the confusion matrix

The model outputs a probability of default p . To make a decision we pick a threshold t and decline everyone with $p \geq t$. Calling a defaulter a “positive”, every applicant falls in one of four boxes (Table II).

TABLE II
THE CONFUSION MATRIX, WITH THE COUNTS OF OUR FINAL MODEL ON THE TEST SPLIT AT THE MONEY THRESHOLD $t = 0.1667$ (61,502 APPLICANTS).

	Declined ($p \geq t$)	Approved ($p < t$)
Defaulted	TP $\approx 2,203$	FN $\approx 2,762$
Repaid	FP $\approx 5,481$	TN $\approx 51,056$

From these four numbers: the *true-positive rate* (TPR, recall) = $TP/(TP + FN)$ is the share of defaulters we catch (44.4%); the *false-positive rate* (FPR) = $FP/(FP + TN)$ is the share of good customers we wrongly decline (9.7%); the *precision* = $TP/(TP + FP)$ is the share of declined people who really would have defaulted; and accuracy is $(TP + TN)/N$ (86.6%).

Why accuracy is misleading here. Only 8.07% of applicants default. A “model” that approves everyone is 91.93%

accurate and catches nobody. Accuracy rewards ignoring the rare class, so we use the measures below instead.

B. Measuring a ranking: ROC curve and AUC

Moving the threshold t from 1 down to 0 traces the *ROC curve*: FPR on the x-axis, TPR on the y-axis. A useless model gives the diagonal; a perfect one goes straight up to the top-left corner. The area under it, *ROC-AUC*, is between 0.5 (random) and 1 (perfect).

In simple terms. ROC-AUC is the probability that the model gives a *randomly chosen defaulter* a higher risk than a *randomly chosen good customer*. Our 0.790 means: pick one of each at random, and 79% of the time the defaulter gets the higher score.

Related numbers: the **Gini** coefficient banks quote is $2 \text{ AUC} - 1$ (0.580 for us); the **KS** statistic is the largest gap between TPR and FPR over all thresholds (0.439); **PR-AUC** is the area under the precision–recall curve, which focuses on the rare defaulters (random guessing gives 0.081, the default rate; we get 0.295).

C. Measuring honesty: calibration

A ranking can be perfect while the probabilities are wrong (for example if every probability were divided by two). For money decisions we need *calibrated* probabilities.

In simple terms. A model is calibrated if, among all applicants it gives about 10%, about 10% really default; among those it gives 30%, about 30% default; and so on.

Two measures: the **Brier score** is the average of $(p_i - y_i)^2$ where $y_i \in \{0, 1\}$ is the true outcome (lower is better); the **expected calibration error** (ECE) sorts applicants into probability bins and averages, weighted by bin size, the gap between the mean predicted probability and the actual default rate in each bin. A *reliability diagram* plots one against the other; a calibrated model lies on the diagonal.

D. Honest evaluation

Splits. If a model is tested on data it learned from, it looks better than it is (*overfitting*). We therefore split the labelled data once: a *train* split to learn from, a *validation* split to make decisions about the system (threshold, calibration, fairness), a *conformal* split reserved for the conformal layer, and a *test* split that is used once at the end.

Cross-validation (CV). To compare models on the train split without wasting data, it is cut into 5 equal *folds*. Each model is trained 5 times, each time on 4 folds and scored on the fifth, so every applicant is scored exactly once by a model that never saw them (*out-of-fold* predictions).

Leakage. Anything learned from data, such as the median used to fill missing values, must be learned inside each training fold only; otherwise information from the scored fold “leaks” in. All such steps live inside scikit-learn pipelines that are refitted per fold.

Uncertainty: the bootstrap. A score from one data set is itself uncertain. The bootstrap [2] resamples the scored applicants with replacement 1,000 times and recomputes the score each time; the middle 95% of those values is the 95% confidence interval (CI). To compare two models we use the *paired* bootstrap: the same resamples for both, and a CI for the *difference*. If that CI does not contain 0, the difference is significant.

Many tests at once. If we run four tests at the 5% level, the chance of at least one false alarm is much larger than 5%. The Holm correction [3] multiplies the smallest p-value by 4, the next by 3, and so on, keeping the overall false-alarm rate at 5%.

E. The winning model family: gradient-boosted trees

A *decision tree* asks a sequence of yes/no questions (“is the external score below 0.3?”) and ends in a leaf that predicts a risk. One deep tree overfits; one shallow tree is too simple. *Gradient boosting* [4] adds many shallow trees one after another; each new tree is fitted to correct the errors the current sum of trees still makes, and its contribution is shrunk by a small *learning rate*. LightGBM [5] and XGBoost [6] are fast implementations; they handle missing values and categories natively.

In simple terms. Boosting is like a team of weak experts where each new expert studies only the cases the team still gets wrong. 1,871 small trees, each adding a small correction, make the final model.

III. RELATED WORK

Credit scoring. Logistic regression on weight-of-evidence (WoE) bins, turned into a points table, is the industry standard because it is transparent [7], [8]. Large benchmarks find that ensembles, especially boosting, rank applicants better [9].

Boosting on tabular data. Gradient boosting [4], [6], [5] is the strongest family on tabular data and still beats deep networks there [10]. Explainable boosting machines (EBM) keep the model additive and readable [11].

Probabilities and decisions. Platt scaling [12] and isotonic regression [13] recalibrate scores; boosted trees are often already close to calibrated [14]. With calibrated probabilities the cost-optimal decision is a threshold set by the costs [15], [16].

Conformal prediction gives prediction sets with a finite-sample guarantee under exchangeability [17], [18]; class-conditional (Mondrian) versions give the guarantee for each class separately.

Explainability. SHAP values [19] are exact and fast for trees (TreeSHAP [20]); LIME [21] fits local surrogate models. Counterfactual explanations [22] should only change features a person can act on [23].

Fairness. Equal opportunity asks for equal true-positive rates across groups [24]; here, equal approval of good customers. Calibration within groups and equal error rates cannot both hold when base rates differ [25], [26]. Mitigations act before training (reweighing [27]), during training (reductions [28]) or after training (group thresholds [24]). Model cards [29] and datasheets [30] document models and data.

IV. DATA

A. Seven linked tables

Home Credit lends to people with little or no credit history. The data has one row per application in the main table and six history tables linked to it by the applicant id SK_ID_CURR (Table III). The label TARGET is 1 if the client had payment difficulties on the loan.

TABLE III
THE SEVEN TABLES (ROWS AFTER CONVERSION TO PARQUET).

Table	Rows	One row is ...
application (train)	307,511	an application, with the label
application (test)	48,744	an application, no label
bureau	1,716,428	a loan at another lender
bureau_balance	27,299,925	a month of such a loan
previous_application	1,670,214	an earlier application here
installments_payments	13,605,401	an instalment payment
POS_CASH_balance	10,001,358	a month of a POS/cash loan
credit_card_balance	3,840,312	a month of a credit card

The model needs exactly one row per applicant, so every history table is summarised per applicant (Sec. V). Coverage differs a lot: 94.8% of applicants have instalment records, 85.7% have bureau records, but only 30.0% have monthly bureau statuses and 28.3% have credit cards.

B. What the exploratory analysis found

Imbalance. 24,825 of 307,511 applicants defaulted (8.07%).

Traps.

- 55,374 rows (18.0%) have DAYS_EMPLOYED = 365243 (that is, 1,000 years in the future). It is a code for “no employment record”, mostly pensioners, who default at 5.40% vs 8.66% for everyone else. We turned it into “missing” plus a flag, because the flag itself is informative. The same code hides in five date columns of the previous applications.
- Four rows have gender “XNA”; they were dropped.
- Income is extremely skewed: median 147,150, maximum 117 million; we use its logarithm.
- 41 of 122 columns are more than half empty (mostly building descriptions), and 64 pairs of numeric columns correlate above 0.9 (the building _AVG/_MODE/_MEDI triples).

Two bugs were caught only because results were checked against the raw data: counting split payments row by row claimed 67% of instalments were underpaid (0% once payments are combined per instalment), and the 365243 code had quietly emptied two previous-loan features.

Strongest single signals. Three “external scores” (normalised scores from outside credit bureaus) have the largest rank correlations with default (−0.166, −0.151, −0.147; a higher score means lower risk).

Groups. Men default at 10.14% vs 7.00% for women; applicants under 30 at 11.44% vs 4.92% at 60 and over. These different *base rates* matter in the fairness section.

Drift preview. The Kaggle test file is a different mix: 0.9% revolving loans vs 9.52% in the training file.

C. Splits

The labelled rows were split once, stratified by the label so each split has the same 8.07% default rate (Table IV). The split file refuses to be overwritten.

TABLE IV
FROZEN SPLITS OF THE LABELLED APPLICATIONS.

Split	Rows	Share	Used for
train	184,503	60%	learning, 5-fold CV
validation	30,751	10%	calibration, money rule, fairness
conformal	30,751	10%	conformal cut-offs only
test	61,502	20%	final evaluation, once

The 48,744 unlabelled Kaggle test applications are never used for evaluation, only for the demo and as a real example of drift.

V. FEATURES

In simple terms. A model cannot read 27 million monthly rows. For each applicant we summarise each history table into a few numbers, such as “how many earlier loans were refused” or “how many months were paid late in the last year”.

Per table. Each history table is aggregated per applicant with its own key: counts, sums, means, maxima, recent-window counts (for example late months in the last year), and late-payment rates. Applicants with no history get count features of 0 and a HAS_HISTORY flag; averages stay missing. Payments are first combined per instalment, and bureau monthly statuses per bureau loan, before summarising.

Engineered ratios. The most useful features combine raw columns: the mean, minimum and maximum of the three external scores; annuity divided by loan amount (how fast the loan is repaid); loan amount above the price of the goods; bureau debt divided by income; total repayment burden; current loan size compared with past loans.

Result. 241 columns: application 121, bureau 31, bureau balance 9, previous applications 32, instalments 22, POS/cash 12, credit cards 14. Gender is kept only for the fairness audit, so the model uses **240 features**.

One code path. The same function builds the application features for training and for the app, so a “what-if” change in the app is processed exactly as in training.

VI. MODELS

A. The model ladder

Fourteen models were trained on the same five folds, from the simplest to the strongest (Table V). In one line each:

- **Dummy:** predicts the average rate for everyone (the floor, AUC 0.5).
- **Naive Bayes:** Bayes’ rule assuming features are independent given the class.
- **Decision tree:** one tree of yes/no questions.
- **KNN:** the risk of the most similar past applicants.
- **Linear / RBF SVM:** a separating line, or a curved boundary via a kernel.
- **Logistic regression:** a weighted sum of features passed through the sigmoid $1/(1 + e^{-z})$.
- **WoE scorecard:** logistic regression on binned features, turned into a points table, as banks use.
- **Random forest (bagging):** many deep trees on random samples, averaged.
- **AdaBoost:** tiny trees, each focusing on the cases the previous ones got wrong.
- **Gradient boosting** (HistGB, XGBoost, LightGBM): Sec. II.
- **EBM:** a boosted model that is a *sum of one curve per feature*, so it can be read like a scorecard.

What the ladder shows.

- The three gradient-boosting libraries are within 0.003 of each other: the *method* matters, not the library.
- Logistic regression is only 0.010–0.014 behind, because the engineered ratios already capture much of the non-linear signal.
- The decision tree shows overfitting clearly: it is best at depth 6 (held-out 0.725); at depth 20 it scores 0.905 on its own training data but only 0.519 on new data.

TABLE V
FIVE-FOLD CV ROC-AUC ON THE TRAIN SPLIT (184,503 APPLICANTS, 240 FEATURES).

Model	ROC-AUC	95% CI
Tuned LightGBM (final)	0.790	0.787–0.793
LightGBM, default settings	0.787	
XGBoost	0.785	
HistGradientBoosting	0.783	
EBM (glass box)	0.776	
Logistic regression	0.776	0.772–0.779
PyTorch MLP (embeddings)	0.772	0.768–0.775
Linear SVM	0.772	0.768–0.775
RBF SVM (Nystroem)	0.764	0.760–0.768
Random forest	0.763	0.759–0.766
AdaBoost	0.757	0.753–0.760
WoE scorecard (30 features)	0.755	0.751–0.758
KNN	0.746	0.742–0.750
Decision tree (depth 8)	0.725	0.721–0.730
Naive Bayes on 30 PCA comp.	0.708	0.704–0.713
LightGBM, main table only	0.761	0.758–0.765
Dummy	0.500	

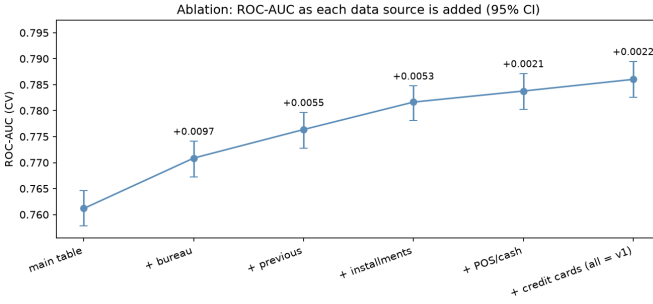


Fig. 1. Ablation: CV ROC-AUC as each history table is added (95% CI).

- Naive Bayes is poor on raw features (0.543). With 240 correlated features it counts the same evidence many times and becomes so overconfident that 82.3% of applicants get a probability of exactly 1.0, which ROC-AUC cannot rank (ties). Ranked by its log-odds it reaches 0.664, and on 30 uncorrelated PCA components 0.708.
- The random forest’s built-in out-of-bag score (0.763) equals its CV score (0.763).

B. Does the loan history help? (ablation)

In simple terms. An ablation removes or adds one ingredient at a time to see how much each one is worth.

Starting from the main table alone (0.761), we added one history table at a time (Fig. 1). The bureau adds the most (+0.0097), then previous applications (+0.0055), instalments (+0.0053), POS/cash (+0.0021) and credit cards (+0.0022): +0.025 in total, with non-overlapping confidence intervals. Each table adds less than the one before, because later tables partly repeat what earlier ones show.

C. Experiment 1: should we “fix” the class imbalance?

With only 8% defaulters, a common recommendation is to rebalance the data: give defaulters a higher weight, throw away

some good customers (*undersampling*), or create synthetic defaulters with **SMOTE** [31], which draws a new point on the line between a defaulter and one of its nearest defaulter neighbours.

Result. Every “fix” made the ranking significantly *worse* (class weights -0.0017 , undersampling -0.0038 , SMOTE -0.0039 ; Table XIII in the appendix). Weights and under-sampling also destroyed calibration: the average predicted probability rose to 0.335 and 0.270, against a true rate of 0.081.

The SMOTE surprise. SMOTE also trained on 1 defaulter per 2 others, yet its average probability stayed honest (0.084). The reason: a point between two real applicants often has a value no real person has, such as 1.4 children. Every synthetic row had at least one such in-between value, and a model could separate synthetic from real defaulters perfectly (ROC-AUC 1.000). The trees learned to recognise fake applicants and gave real ones roughly their real rate. **Decision:** train on the natural 8% rate.

D. Experiment 2: tuning

LightGBM has settings (*hyperparameters*) such as the number of leaves per tree or the learning rate. We compared three ways of searching them, each with the same budget of 27 trials, scored on fold 1 with early stopping:

- **grid search** tries every combination on a fixed grid;
- **random search** samples combinations at random [32];
- **TPE** (Optuna [33], [34]) builds a probability model of which settings score well and samples where good results are likely.

Best scores: grid 0.7927, random 0.7928, TPE 0.7942 (default settings 0.7894). The differences between methods are smaller than the noise on one fold. In full 5-fold CV the tuned model scores 0.7899 vs 0.7870, a significant +0.0029, but smaller than the +0.0048 seen on the fold used for tuning; choosing the best of many trials on one fold makes that fold look too good (the “winner’s curse”). The final model has 1,871 trees of 17 leaves, at least 426 applicants per leaf, and a learning rate of 0.022: many small, heavily regularised trees.

E. Neural networks

A PyTorch MLP with learned embeddings for categories (two hidden layers of 256 and 128 units, batch normalisation, 30% dropout, early stopping) reached 0.772, 0.018 behind LightGBM. A 50/50 blend of the two was *worse* than LightGBM alone (-0.0040): their predictions correlate at 0.887, so the network mostly makes the same mistakes while being weaker. This matches the finding that trees still beat deep learning on tabular data [10].

F. Algorithms from scratch

To show the mathematics is understood, seven algorithms were written in NumPy and checked against scikit-learn on the same data (Table X in the appendix):

- **Logistic regression by Newton’s method.** Minimising the log-loss plus an L2 penalty is the same as finding the

most probable weights under a Gaussian prior (*MAP*). Newton’s method uses the curvature and converges in about 10 steps. The **Laplace** version approximates the posterior by a Gaussian around that optimum, giving each applicant an uncertainty. Finding: uncertainty is not risk; the 10% most uncertain applicants default at 6.4% vs 8.5%.

- **Gaussian Naive Bayes**, **PCA** via the singular value decomposition, and **Fisher LDA** (the direction that best separates the classes).
- **K-means++** (careful random starts, then alternate “assign to nearest centre” and “move centre to the mean”) and a **Gaussian mixture fitted by EM**, which never lowered the likelihood in 50 rounds.
- An **MLP with hand-written backpropagation**, whose gradients match numerical derivatives to 2×10^{-7} .

VII. FINAL EVALUATION ON THE TEST SPLIT

In simple terms. The test split is like the final exam paper: kept sealed while studying, opened once. Everything that could be chosen was chosen before it was opened.

What was fixed in advance. The models: the final LightGBM, its reweighed version (Sec. X), and logistic regression, the EBM and the scorecard refitted once on the whole train split. The tests: the final model against each other model. The decision rules: the money threshold and the conformal cut-offs, unchanged.

Two significance tests.

- The **paired bootstrap** on ROC-AUC asks: does the final model *rank* better?
- The **McNemar test** [35] asks: does it make better *decisions* at the threshold? It looks only at applicants where exactly one of the two models is right. If the models were equally good, each such case would be a coin flip; a lopsided count is evidence of a real difference.

Each family of four tests is Holm-corrected. Separately, **Dieterich’s 5×2cv t-test** [36] (five repetitions of a 2-fold split) compares LightGBM with logistic regression; it retrains models, so it uses the train split only.

Results (Table VI, Fig. 2).

- Every model scores within 0.005 of its CV estimate: neither model choice nor tuning overfitted the folds. The final model: ROC-AUC 0.790, KS 0.439, PR-AUC 0.295, Brier 0.0653, ECE 0.0018.
- LightGBM *ranks* significantly better than all three reference models (all $p < 0.001$ after Holm). The 5×2cv test agrees for logistic regression (+0.011, $t = 5.38$, $p = 0.003$, positive in all 10 halves).
- On *decisions* the gap almost vanishes: 86.6% correct vs 86.2–86.4%. McNemar is significant only against logistic regression ($p = 0.0005$); against the EBM and the scorecard $p = 0.066$. Most of the ranking advantage lies away from the one threshold that matters.
- The **price of transparency** is 0.009 ROC-AUC for the EBM, with indistinguishable decision accuracy.

TABLE VI
FROZEN TEST RESULTS (61,502 APPLICANTS, 8.07% DEFAULT). Δ AUC IS THE FINAL MODEL’S ADVANTAGE (PAIRED BOOTSTRAP).

Model	ROC-AUC [95% CI]	PR-AUC	Brier	Δ AUC
LightGBM (final)	0.790 [0.784, 0.797]	0.295	0.0653	–
LightGBM, reweighed	0.790 [0.784, 0.797]	0.294	0.0653	+0.000
EBM (glass box)	0.781 [0.774, 0.788]	0.273	0.0663	+0.009*
Logistic regression	0.778 [0.771, 0.785]	0.272	0.0664	+0.013*
WoE scorecard	0.757 [0.751, 0.765]	0.239	0.0679	+0.033*

* significant after Holm correction. Final model: KS 0.439, Gini 0.580, ECE 0.0018.

Learning curves: would more data help? We trained on 2% to 100% of four folds and scored on the fifth (Fig. 3). Logistic regression’s training and held-out curves have met at a low level: it is limited by its simple form (*high bias*), so more data barely helps (0.776 $\rightarrow$ 0.778 from half to all). LightGBM’s training score is still far above its held-out score (*variance*), and its held-out score is still rising (0.784 $\rightarrow$ 0.794): more data would help the boosted model, not the linear one.

VIII. FROM PROBABILITY TO DECISION

A. Are the probabilities honest? (calibration check)

On the validation split the raw model was already calibrated: mean predicted 8.03% vs 8.07% actual, Brier 0.06529, ECE 0.0024. We still tried two standard fixes, fitted with five-fold cross-fitting inside the validation split so no applicant is scored by a fix that saw them: **Platt scaling** [12] fits a logistic curve to the scores, and **isotonic regression** [13] fits a step function that only goes up. A fix was kept only if its Brier improvement had a 95% CI entirely below 0. Platt changed the Brier score by +0.000005 (CI -0.000016 to $+0.000027$), and isotonic lowered ROC-AUC (its steps create ties). **Decision: no calibrator.** Boosted trees with a log-loss objective are often already calibrated [14], and avoiding the imbalance “fixes” kept it that way. On test: 7.99% predicted vs 8.07% actual.

One caveat matters later: the model is calibrated *overall* but not *within gender* (men 9.3% predicted vs 10.4% actual; women 7.4% vs 6.9%).

B. The money rule

In simple terms. Lend if the expected gain is positive.

Suppose approving a loan of size A earns a margin m (say 10%) if it is repaid, and loses a share LGD (“loss given default”, say 50%) if it is not. With default probability p , the expected profit is

$$\mathbb{E}[\text{profit}] = (1 - p)mA - p\text{LGD}A, \quad (1)$$

which is positive exactly when

$$p < t^* = \frac{m}{m + \text{LGD}} = \frac{0.10}{0.10 + 0.50} = 0.1667. \quad (2)$$

Worked example. A loan of 500,000 earns 50,000 if repaid and loses 250,000 if not. At $p = 0.10$ the expected profit is $0.9 \times 50,000 - 0.1 \times 250,000 = +20,000$: approve. At $p = 0.20$

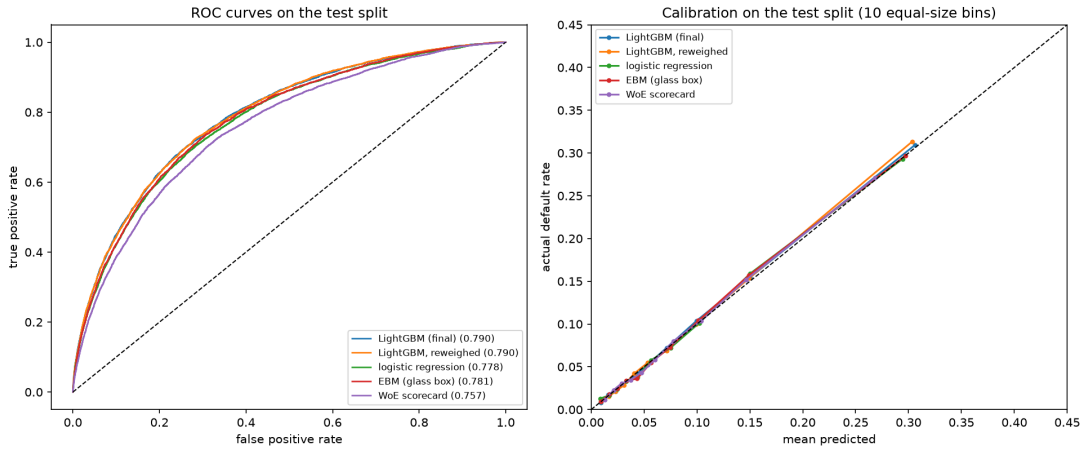


Fig. 2. Test split: ROC curves (left) and calibration in ten equal-size bins (right) for the five frozen models. All five lie on the diagonal of the calibration plot.

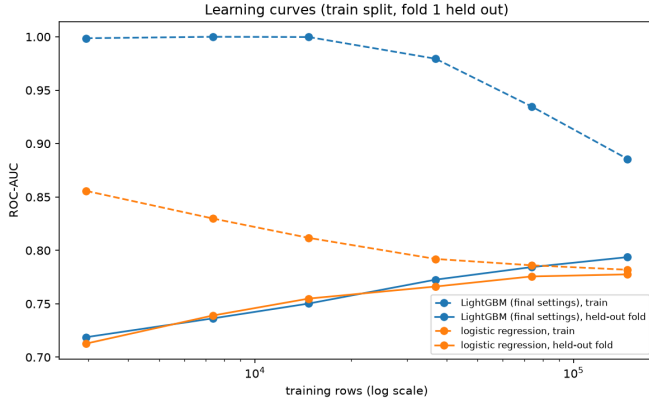


Fig. 3. Learning curves (train split, fold 1 held out).

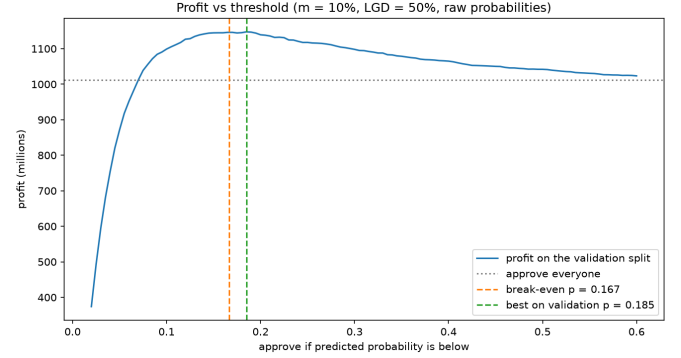


Fig. 4. Validation profit vs the approval threshold. The break-even threshold (0.167) is within 0.3% of the best one.

it is $0.8 \times 50,000 - 0.2 \times 250,000 = -10,000$: decline. The rule only works if p is a real probability, which is why calibration was checked first.

Result. On validation the rule approves 87.4%, declines 44.5% of defaulters but only 9.8% of good customers, and earns 1,144M vs 1,011M for approving everyone (+13.1%). Searching for the best threshold on validation finds 0.185, which earns only 0.3% more (Fig. 4): decision theory gets the threshold right without any search. On test the rule earns +13.9%. The threshold depends on the assumed margin and loss: across plausible values the approval rate ranges from 54% to 99%.

C. A guarantee instead of a guess: the conformal layer

The money rule always gives an answer, even when the model is unsure. We want a rule that *promises* limits on both mistakes and sends unclear cases to a person.

In simple terms. For each applicant we build a *set* of answers the model cannot rule out: {repay}, {default}, or both. If only “repay” is left we approve; if only “default” is left we decline; if both are left the model is unsure and a person decides. The cut-offs are set on held-out data so that at most 10% of real defaulters end up approved and at most 10% of good customers end up declined.

How it works. Each possible answer gets a *non-conformity score*: how strange that answer would be. For “repay” the score is p (a repayer with a high p is strange); for “default” it is $1 - p$. On the *conformal* split (30,751 applicants never used for anything else) we look at each class separately. Among the n applicants who really repaid, the cut-off q_{repay} is the $[(n + 1)(1 - \alpha)]$ -th smallest score, so that about $1 - \alpha$ of repayers score below it; q_{default} is computed the same way among defaulters. A new applicant’s set contains “repay” if $p \leq q_{\text{repay}}$ and “default” if $1 - p \leq q_{\text{default}}$. We use $\alpha = 10\%$.

The cut-offs we got. $q_{\text{repay}} = 0.1655$ and $q_{\text{default}} = 0.9645$, which gives a simple rule: **approve if $p < 0.0355$, decline if $p > 0.1655$, refer in between.**

Worked example (three real demo applicants):

- $p = 0.019$: “repay” is in the set ($0.019 \leq 0.1655$), “default” is not ($1 - 0.019 = 0.981 > 0.9645$). Only

“repay”: **approve**.

- $p = 0.070$: both conditions hold. Both answers possible: **refer**.
- $p = 0.227$: “repay” is out ($0.227 > 0.1655$), “default” is in. **Decline**.

The guarantee. If new applicants are *exchangeable* with the conformal split (roughly: drawn from the same population), the share of defaulters whose set wrongly excludes “default” is at most about α , and the same for repayers [17], [18]. It needs no assumption about the model being right.

Does it hold? Over 100 random half-splits of the conformal split, coverage averaged 0.901 for repayers and 0.899 for defaulters. On the untouched test split it was 0.902 for both, so **9.8%** of defaulters were auto-approved and 9.8% of good customers auto-declined, as promised.

The price. Certainty costs automation: on test 40.5% are approved, **46.9% referred** and 12.6% declined. Fig. 5 shows the trade-off: a smaller α (a stricter promise) sends more people to the refer zone.

Why one cut-off per class. With a single cut-off for everyone the promise holds only *on average*, and the 92% majority of repayers dominates it: it would have auto-approved 82% of defaulters. Note also that the decline line (0.1655) almost equals the money threshold (0.1667).

IX. EXPLANATIONS

A. SHAP: splitting a prediction into reasons

In simple terms. Start from the risk of a typical applicant. Each feature of this applicant then pushes the risk up or down. The pushes add up exactly to this applicant’s score. The biggest upward pushes are the reasons for a decline.

SHAP values [19] come from cooperative game theory: each feature gets its fair share of the difference between this prediction and the average prediction, averaged over all orders in which features could be added. For tree models TreeSHAP computes them exactly and fast [20]; LightGBM provides them directly (in log-odds units).

Worked example. The declined applicant from Sec. VIII ($p = 0.227$; a typical applicant gets 0.050) has these top upward pushes, turned into *reason codes*:

- “External credit scores (lowest) is 0.031, which raised the estimated risk.”
- “External credit scores (average) is 0.271, ...”
- “Previous applications: refused share is 0.500, ...”
- “Previous applications: total cost mean is 0.538, ...”

Lenders must give declined customers their main reasons (“adverse action notices” in the US); the app shows exactly these sentences.

B. Which features matter overall?

Three views of global importance were compared: **gain** (how much each feature improved the trees during training), **permutation** (how much held-out ROC-AUC drops when that feature’s values are shuffled) and **mean |SHAP|**. All three put the average external score first, and the top five by SHAP are the average, highest and lowest external score, annuity $\div$ loan

amount and employer type. Gain and SHAP rankings agree closely (rank correlation 0.945), but permutation and SHAP only at 0.42: when two features carry the same information, shuffling one barely hurts, so permutation spreads the credit. 85 of the 240 features have negative permutation importance, candidates for a simpler model.

Summing |SHAP| per source table (Fig. 6), the six history tables carry **53.3%** of the model’s evidence, matching the ablation.

Partial dependence (PDP). Averaging predictions while one feature is varied shows its average effect: risk falls from about 9% to 5% as the average external score rises; annuity $\div$ loan amount jumps at particular loan terms; age looks flat on its own because its signal is shared with correlated features. A flat PDP does not mean “no effect”.

C. Is LIME a good substitute?

LIME [21] explains one prediction by fitting a simple linear model to the model’s outputs on random perturbations around the applicant. Its top five reasons shared on average 2.1, 3.2 and 4.1 features with SHAP’s top five for an approved, a referred and a declined example, and changed between random seeds (Jaccard similarity 0.62–0.74). The clearer the case, the more stable LIME is. Since TreeSHAP is exact and deterministic for this model, the app uses SHAP.

D. What would change the decision? (counterfactuals)

In simple terms. Instead of “you were declined because ...”, tell the applicant “if you borrowed X% less, you would not be declined”.

A counterfactual explanation [22] is the smallest change that flips the decision. It is only useful if it changes things a person can act on [23], so we search only two options: a *smaller loan* (loan and annuity together, in 5% steps) and a *higher income* (10% steps, up to $4\times$). Age, gender and the external scores are never changed. Every candidate goes through the real feature code, so all ratios are rebuilt.

Results. The declined example leaves the decline zone only when the loan is cut to 25% of the requested amount (p : $0.227 \rightarrow 0.162$); no income up to $4\times$ is enough. Over all 112 declined demo applicants, **borrowing less** moves 77.7% out of the decline zone (median: 35% less), while a higher income works for only 20.5% (median $\times 1.7$). For this model the size of the loan relative to the repayment matters far more than income.

E. The price of transparency

If a regulator demanded a fully readable model: the WoE scorecard costs 0.031 ROC-AUC (CV), the EBM only 0.010 (0.009 on test, with the same decision accuracy as LightGBM). The EBM would be the choice.

X. FAIRNESS

A. Definitions

We compare groups (gender: F/M; age bands: <30, 30–45, 45–60, 60+) at the money rule. For each group g :

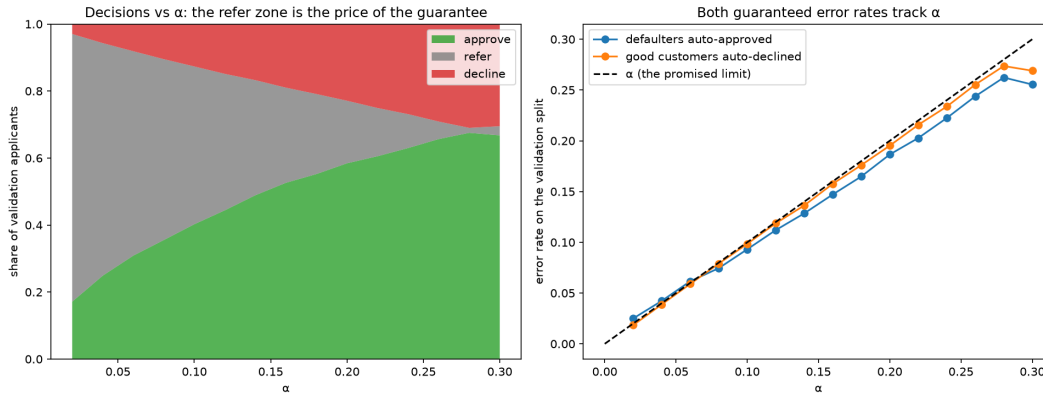


Fig. 5. Conformal layer on validation: decision shares (left) and the two guaranteed error rates (right) as α varies. Both error rates track the promised α .

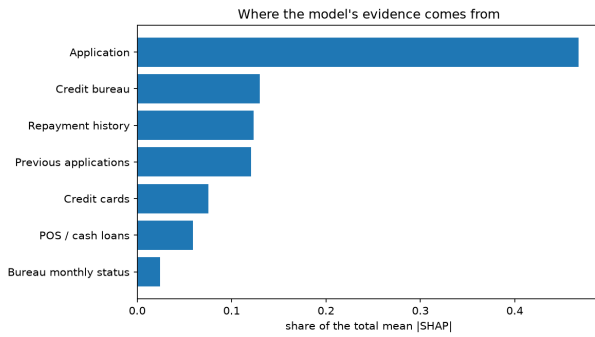


Fig. 6. Share of the total mean |SHAP| per source table.

- **approval rate**;
- **good customers declined** = FPR_g , the share of people who would have repaid but were declined; the harm of a wrong rejection;
- **defaulters declined** = TPR_g .

Fairness measures compare groups:

- **demographic parity difference**: the gap in approval rates;
- **equal opportunity gap** [24]: the gap in good customers declined; **our main measure**, because it protects the people a wrong rejection harms;
- **equalized odds difference**: the larger of the FPR and TPR gaps;
- **disparate impact ratio**: lowest approval rate $\div$ highest; the US “80% rule” flags values below 0.8.

B. Audit

Table VII gives the validation audit (test values repeat these within about 1.5 points).

- **Gender**: good men are declined at 12.6% vs 8.3% for good women, an equal opportunity gap of **4.3 points** against men; disparate impact 0.94.
- **Age**: larger gaps: 18.0% of good customers under 30 are declined vs 3.1% at 60+; disparate impact 0.81 on validation and **0.796 on test**, just under the 80% line.

TABLE VII

FAIRNESS AUDIT ON VALIDATION AT THE MONEY RULE (PERCENT).

Group	Default rate	Predicted	Approved	Good declined
Women	6.9	7.4	89.4	8.3
Men	10.4	9.3	83.7	12.6
Under 30	11.7	11.5	77.8	18.0
30–45	9.2	8.9	85.2	11.5
45–60	6.5	6.7	91.2	6.7
60+	4.5	4.7	96.3	3.1

- **The conformal promise holds per class, not per group**: 23% of defaulters aged 60+ are auto-approved (target 10%), and 18% of good customers under 30 are auto-declined.

C. Proxy test: does leaving gender out hide it?

Gender is not a model input. But a LightGBM trained to predict gender from the other 240 features reaches ROC-AUC **0.911**, mainly through occupation, employer type and car ownership. Deleting a column does not delete the information. The risk score itself, however, carries little gender (0.568 when used to predict it), so the gap comes mostly from real differences in default rates.

D. Why perfect fairness is impossible here

For any decision rule and any group, Chouldechova [26] showed that

$$\text{FPR} = \frac{b}{1-b} \cdot \frac{1-\text{PPV}}{\text{PPV}} \cdot (1-\text{FNR}), \quad (3)$$

where b is the group’s default rate, PPV the precision of the declines and FNR the share of defaulters missed. With our numbers it reproduces both groups’ FPRs exactly (0.0833 and 0.1263). Now suppose women had the same PPV (0.306) and the same FNR (0.521) as men. With women’s default rate $b = 0.0687$ the identity forces their FPR to 0.080, against men’s 0.126.

TABLE VIII
GENDER MITIGATIONS ON VALIDATION: GAP IN GOOD CUSTOMERS
DECLINED (POINTS) AND PROFIT RELATIVE TO THE FINAL MODEL
(1,144.2M).

Method	Gap	Profit	Uses gender
Final model	4.3	—	no
Reweighting (retrained)	1.7	−1.2M	no
ExponentiatedGradient	1.7	−1.3M	no
Group thresholds, gap ≤ 1 pt	0.8	−1.3M	yes
Group thresholds, gap ≤ 0.25 pt	0.1	−3.6M	yes
Group thresholds, no limit	9.4	+1.5M	yes

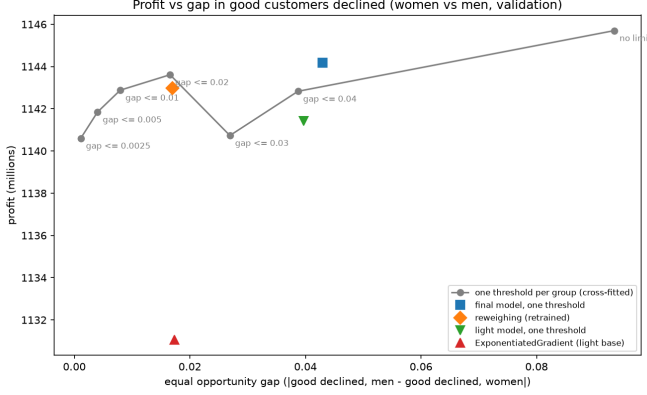


Fig. 7. Profit vs the gender gap in good customers declined (validation). Differences of a few million are within noise.

In simple terms. When two groups default at different rates, a rule cannot be equally precise *and* make equal mistakes for both. One kind of fairness has to give way; choosing which is a policy decision, not a technical one [25].

E. Three mitigations

- 1) **Reweighting (before training)** [27]: give each (group, label) cell the weight $P(g)P(y)/P(g, y)$, so that in the weighted data gender and default look independent, then retrain. Gender is needed only for training.
- 2) **ExponentiatedGradient (during training)** [28]: Fairlearn trains a mixture of models that minimises the money cost subject to an FPR gap of at most 0.01. Its decisions are randomised (a mix of models).
- 3) **Group thresholds (after training)** [24]: one threshold per gender, the most profitable pair whose gap is within a limit. Fairlearn’s ThresholdOptimizer was not used because it maximises accuracy, which at 8% defaulters means approving almost everyone.

Results (Table VIII, Fig. 7). Reweight closes 60% of the gap for 0.1% of profit. ExponentiatedGradient reaches the same gap for 1.1% of profit and gives randomised decisions: two identical applicants could get different answers. Group thresholds can close the gap fully, but they need gender *when deciding*, which many credit laws forbid. Strikingly, thresholds chosen for profit alone *double* the gap: profit pushes away from fairness.

Choice and its cost. We chose reweighting. On test it kept the same ROC-AUC (difference 0.0001, $p = 0.86$) and decision accuracy, and cut the gap from 4.6 to 2.0 points for 0.55% of profit. The cost is the impossibility result in practice: calibration within gender got worse (men 8.7% predicted vs 10.2% actual). Because the conformal cut-offs were fitted and checked on test only for the original model, the app keeps that model, and the reweighted one is documented as a tested alternative.

XI. UNSUPERVISED LEARNING

In simple terms. Without using the labels, are there natural groups of applicants, and do those groups differ in risk?

Clustering groups similar applicants. On 7 standardised key features (log income, log loan, log annuity, family size, age, years employed, mean external score) of 20,000 train applicants we compared four methods: **K-means** (choose k by the elbow and the *silhouette*, which measures how much closer each point is to its own cluster than to the next one, from -1 to 1), a **Gaussian mixture** (choose the number of components by BIC, which rewards fit and penalises complexity), **Ward hierarchical clustering** (repeatedly merge the two closest groups; a dendrogram shows the merges) and **HDBSCAN** [37] (clusters are dense regions; sparse points are labelled noise).

Results.

- The applicants form **one blob**: the best silhouette is only 0.19 ($k = 2$, which simply splits large loans from small ones). Ward agrees only partly (ARI 0.52); HDBSCAN finds four pockets and calls 20% noise.
- **BIC can be fooled**: it prefers 9 Gaussian components, but 8 of them have zero variance in family size: they sit on whole numbers (point masses), whose likelihood can grow without limit. The silhouette (0.025) shows these clusters are not real.
- **Clusters are not risk**: using a cluster’s default rate as a score gives ROC-AUC 0.50–0.55 for every method, against 0.79 for the model.
- **Maps**: the 43 numeric building columns need only 8 PCA components for 90% of their variance (they repeat each other); one Fisher LDA direction ranks risk at 0.714; t-SNE shows defaulters spread everywhere, with no separate island.

Risk personas. Clustering the 3,897 validation applicants in the decline zone by their *SHAP vectors* groups people who are risky *for the same reasons* [20] (Fig. 8): “low external scores” (1,637 applicants, 31.0% default), “maxed-out credit card” (652, 30.1%) and “borderline, mixed reasons” (1,608, 25.4%). The structure is soft (silhouette 0.099) but tells a reviewer what to look at.

XII. THE SYSTEM

A. Architecture

Fig. 9 shows the whole pipeline. At run time a *scoring service* loads the frozen model and a small decision file (the money threshold and the conformal cut-offs). For an applicant

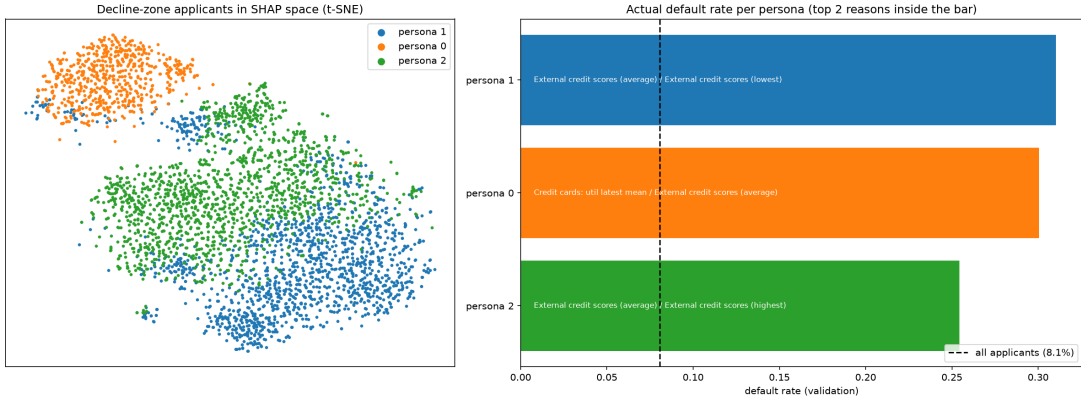


Fig. 8. Risk personas: decline-zone applicants clustered by their SHAP values (t-SNE map, left) and each persona’s actual default rate (right). Persona 1 = low external scores, persona 0 = maxed-out credit card, persona 2 = borderline.

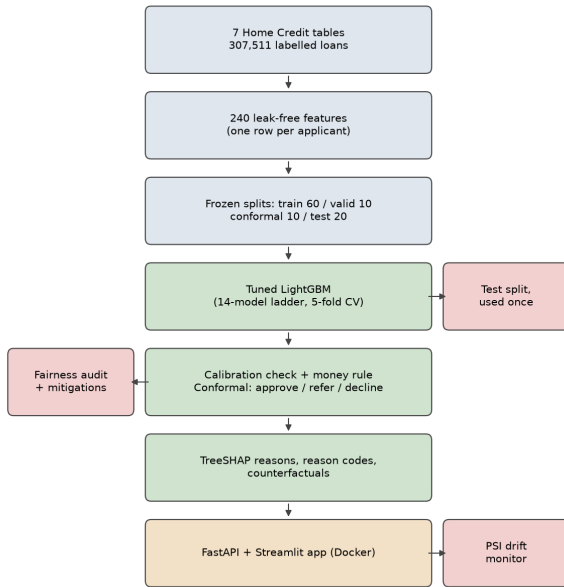


Fig. 9. System architecture.

and up to eight “what-if” changes (income, loan amount, annuity, age, years employed, three external scores) it rebuilds the features with the training code, predicts, decides, and explains with TreeSHAP.

API. FastAPI exposes the service; Pydantic validates every input (for example, external scores must be between 0 and 1, and years employed cannot exceed age minus 14). A call looks like:

```
POST /score
{"applicant_id": 101383,
 "changes": {"income": 250000}}
```

and returns the probability, the decision (with the money rule’s suggestion), the reason codes and the top reasons both ways. GET /applicants/{id}/counterfactuals returns

the smallest change that reaches the next better decision.

App. A Streamlit page with three tabs: *Score + why* (probability and a reasons chart), *Decision* (approve/refer/decline with an explanation and the reason codes) and *What-if* (sliders, and a button that searches for the smallest change).

Public demo without publishing data. The competition rules forbid sharing the data, and we chose not to publish the real model. The public demo (link on the title page) therefore uses 300 *made-up* applicants generated with every real column, and a *stand-in* model trained only on made-up applicants, clearly labelled as such. It shows how the system works; the real model’s results are the ones in this report. The service runs in one Docker image; 133 unit tests run on synthetic data in GitHub Actions on every push.

B. Monitoring drift with PSI

In simple terms. The guarantees above assume new applicants look like the old ones. The population stability index (PSI) checks this without waiting for outcomes, by comparing how applicants are spread over bins today vs at training time.

For a reference sample and a new sample, bin the reference (deciles for numbers, one bin per category) and compare the shares p_k (reference) and q_k (new):

$$\text{PSI} = \sum_k (q_k - p_k) \ln \frac{q_k}{p_k}. \quad (4)$$

Below 0.1 is stable, 0.1–0.25 “watch”, above 0.25 a shift.

Worked example. Contract type: revolving loans are 9.52% of training applicants but 0.9% of the later Kaggle applications; cash loans 90.48% vs 99.1%. Then $\text{PSI} = (0.009 - 0.0952) \ln \frac{0.009}{0.0952} + (0.991 - 0.9048) \ln \frac{0.991}{0.9048} \approx 0.203 + 0.008 = 0.211$: the watch zone.

Results (Fig. 10).

- Between our own splits, PSI is at most 0.002 everywhere: the monitor stays quiet when nothing changed.
- For the later Kaggle applications the *score* is stable (0.008), but annuity ÷ loan amount, the fourth most important feature, has shifted strongly (0.997): the new

loans are shorter (median about 16 instalments instead of 20).

- Simulated on the demo applicants: 20% lower incomes trip the income alarm (0.89) but barely move the score (0.003); 20% lower external scores shift the score (0.254) and cut auto-approvals from 44% to 25%.

A score alert means the conformal cut-offs no longer carry their guarantee and must be refitted on recent labelled applicants. PSI sees only inputs and scores; it cannot see *concept drift* (the same applicant becoming riskier), which needs outcomes that arrive months later.

XIII. ETHICS AND LIMITATIONS

Which fairness. With different base rates, equal error rates and calibration within groups cannot both hold; we chose equal opportunity for good customers and measured what it costs. Age is a model input: it is predictive, and older applicants are favoured, but the approval ratio by age is close to the 80% rule and the conformal promise does not hold within age groups. A group-conditional conformal layer would fix this but needs the group when deciding.

Reject inference. Outcomes exist only for loans Home Credit approved in the past, so the training data is a biased sample of all applicants: people the old policy rejected never show whether they would have repaid. The model is therefore trustworthy for applicants who look like past approvals, and its scores for unusual applicants rest on extrapolation. Semi-supervised reject-inference methods use the unlabelled applicants to correct this, but without outcomes for rejected people their gains cannot be verified; we state the limitation rather than claim a correction.

Privacy. The data stays outside the repository; only aggregate tables and figures are committed. The API validates every input and stores nothing. The public demo contains no real person and no real model.

Inherited bias. The external scores are the strongest inputs and their origin is unknown, so any bias in them is inherited.

Human in the loop. The refer zone keeps a person in charge of the 47% of cases where the model is unsure, and every automatic decision comes with reasons and a counterfactual, so it can be contested.

Assumptions. Profit depends on the assumed margin and loss; the conformal guarantee assumes exchangeability; one lender and one period (data released in 2018). The datasheet and the model card [30], [29] are in the repository.

XIV. CONCLUSION AND FUTURE WORK

A carefully engineered gradient-boosting model reached ROC-AUC 0.790 on untouched test data, exactly its cross-validation estimate, but the larger contribution is everything around it. Its probabilities were honest without help; a break-even rule turned them into 13.9% more profit; a class-conditional conformal layer kept its error promise on test; every decision comes with reasons and an actionable counterfactual; and the costs of fairness were measured instead of assumed.

Five lessons. (1) Class-imbalance “fixes” hurt a model that is already calibrated. (2) Ranking gains do not always become decision gains. (3) Fairness interventions move errors between groups and cost calibration. (4) Structure in the inputs is not structure in the risk. (5) A promise (conformal) is more useful than a single number, and it costs automation.

Future work. Refit the conformal layer per group where the law allows; evaluate reject inference with outcome data; and treat credit-limit setting as a sequential decision problem with reinforcement learning [38], where each limit change affects both repayment and the data collected next.

REFERENCES

- [1] Home Credit Group, “Home credit default risk,” Kaggle competition, <https://www.kaggle.com/c/home-credit-default-risk>, 2018.
- [2] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.
- [3] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [4] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [5] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, 2017.
- [6] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016.
- [7] L. C. Thomas, D. B. Edelman, and J. N. Crook, *Credit Scoring and Its Applications*. SIAM, 2002.
- [8] N. Siddiqui, *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring*. Wiley, 2006.
- [9] S. Lessmann, B. Baesens, H.-V. Seow, and L. C. Thomas, “Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research,” *European Journal of Operational Research*, vol. 247, no. 1, pp. 124–136, 2015.
- [10] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on typical tabular data?” in *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2022.
- [11] H. Nori, S. Jenkins, P. Koch, and R. Caruana, “InterpretML: A unified framework for machine learning interpretability,” *arXiv preprint arXiv:1909.09223*, 2019.
- [12] J. C. Platt, “Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods,” in *Advances in Large Margin Classifiers*. MIT Press, 1999.
- [13] B. Zadrozny and C. Elkan, “Transforming classifier scores into accurate multiclass probability estimates,” in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2002.
- [14] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in *Proc. Int. Conf. Machine Learning*, 2005.
- [15] C. Elkan, “The foundations of cost-sensitive learning,” in *Proc. Int. Joint Conf. Artificial Intelligence*, 2001.
- [16] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [17] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. Springer, 2005.
- [18] A. N. Angelopoulos and S. Bates, “A gentle introduction to conformal prediction and distribution-free uncertainty quantification,” *arXiv preprint arXiv:2107.07511*, 2021.
- [19] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, 2017.
- [20] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, “From local explanations to global understanding with explainable AI for trees,” *Nature Machine Intelligence*, vol. 2, pp. 56–67, 2020.
- [21] M. T. Ribeiro, S. Singh, and C. Guestrin, “Why should I trust you?: Explaining the predictions of any classifier,” in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016.

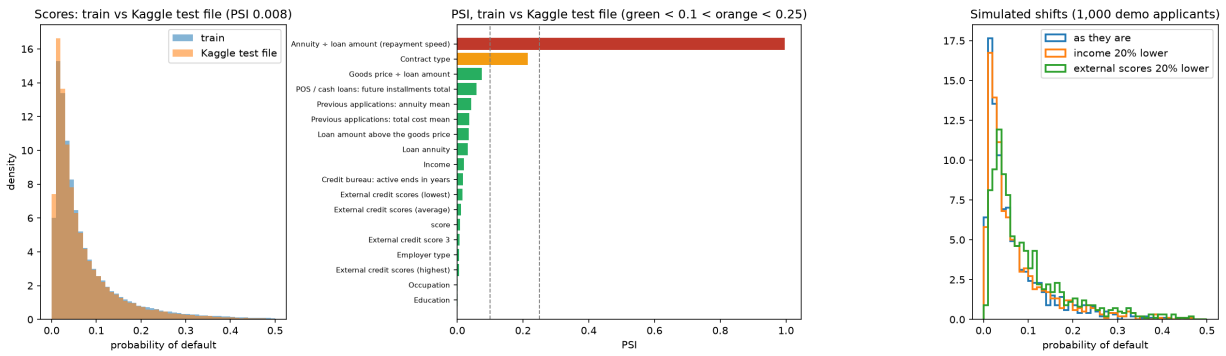


Fig. 10. Drift monitoring: scores on train vs the later Kaggle applications (left), PSI of the watched features (middle), and simulated shifts (right).

- [22] S. Wachter, B. Mittelstadt, and C. Russell, "Counterfactual explanations without opening the black box: Automated decisions and the GDPR," *Harvard Journal of Law & Technology*, vol. 31, no. 2, pp. 841–887, 2018.
- [23] B. Ustun, A. Spangher, and Y. Liu, "Actionable recourse in linear classification," in *Proc. Conf. Fairness, Accountability, and Transparency*, 2019.
- [24] M. Hardt, E. Price, and N. Srebro, "Equality of opportunity in supervised learning," in *Advances in Neural Information Processing Systems*, 2016.
- [25] J. Kleinberg, S. Mullainathan, and M. Raghavan, "Inherent trade-offs in the fair determination of risk scores," in *Proc. Innovations in Theoretical Computer Science Conf.*, 2017.
- [26] A. Chouldechova, "Fair prediction with disparate impact: A study of bias in recidivism prediction instruments," *Big Data*, vol. 5, no. 2, pp. 153–163, 2017.
- [27] F. Kamiran and T. Calders, "Data preprocessing techniques for classification without discrimination," *Knowledge and Information Systems*, vol. 33, no. 1, pp. 1–33, 2012.
- [28] A. Agarwal, A. Beygelzimer, M. Dudík, J. Langford, and H. Wallach, "A reductions approach to fair classification," in *Proc. Int. Conf. Machine Learning*, 2018.
- [29] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, "Model cards for model reporting," in *Proc. Conf. Fairness, Accountability, and Transparency*, 2019.
- [30] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. D. III, and K. Crawford, "Datasheets for datasets," *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [31] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [32] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.
- [33] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2019.
- [34] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Advances in Neural Information Processing Systems*, 2011.
- [35] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [36] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.
- [37] R. J. G. B. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Proc. Pacific-Asia Conf. Knowledge Discovery and Data Mining*, 2013.
- [38] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

APPENDIX A

GLOSSARY

TABLE IX
GLOSSARY OF THE TERMS USED IN THIS REPORT, IN ONE LINE EACH.

Term	Meaning
Default	The borrower had payment difficulties (TARGET = 1); 8.07% of applicants.
Probability of default (p)	The model's output: the estimated chance this applicant defaults.
Feature / label	An input number about the applicant / the outcome we try to predict.
Train / validation / test split	Data to learn from / to tune the system on / to evaluate once at the end.
Cross-validation (CV)	Train on 4 of 5 folds, score the 5th, rotate; every applicant is scored by a model that never saw it.
Overfitting	Learning the training data by heart, so new data scores much worse.
Leakage	Information from the evaluation data sneaking into training; makes results look too good.
ROC-AUC	Chance that a random defaulter gets a higher risk than a random good customer (0.5 random, 1 perfect).
Gini, KS	$2 \cdot \text{AUC} - 1$; the largest gap between the shares of defaulters and of good customers above a threshold.
PR-AUC	Area under the precision–recall curve; focuses on the rare defaulters (random = 0.081).
Calibration	Predicted probabilities match observed frequencies (of those given 10%, about 10% default).
Brier score, ECE	Mean squared error of the probabilities; average gap between predicted and actual rates per bin.
Bootstrap CI	Resample the data 1,000 times to see how much a score could vary; the middle 95% is the interval.
Paired bootstrap	The same resamples for two models; a CI for their difference.
McNemar test	Compares two models on the cases where exactly one of them is right.
Holm correction	Adjusts p-values when several tests are run, so false alarms stay at 5% overall.
Gradient boosting	Many small trees added one by one, each correcting the errors left by the previous ones.
Hyperparameter	A setting chosen before training (e.g. number of leaves), not learned from the data.
SMOTE	Creates synthetic minority examples between real ones; here it hurt.
Break-even threshold	Approve if $p < m/(m + \text{LGD})$: where the expected profit of lending turns positive.
LGD	Loss given default: the share of the loan lost when a borrower defaults (assumed 50%).
Conformal prediction	Turns a score into a set of possible answers with a guaranteed error rate.
Coverage	The share of applicants whose set contains their true outcome (target $1 - \alpha = 90\%$).
Exchangeability	New applicants look like (come from the same population as) the calibration applicants.
Refer zone	Applicants for whom both answers are possible; a person decides.
SHAP value	One feature's push on one prediction; all pushes add up to the prediction.
Reason code	A sentence built from the largest risk-raising SHAP values.
Counterfactual	The smallest actionable change (here: a smaller loan or a higher income) that changes the decision.
LIME	Explains one prediction with a simple local linear model fitted to random perturbations.
PDP	The model's average prediction as one feature is varied.
Equal opportunity gap	Difference between groups in the share of good customers wrongly declined (our main fairness measure).
Disparate impact ratio	Lowest group approval rate divided by the highest; below 0.8 is flagged ("80% rule").
Proxy	A feature that reveals a protected attribute (gender) even when that attribute is removed.
Reweighting	Training weights that make group and outcome look independent.
Silhouette	How much closer a point is to its own cluster than to the next one (−1 to 1).
PSI	Population stability index: how much a distribution has moved (above 0.25 = shift).
Drift	New applicants (data drift) or their risk (concept drift) differ from the training period.
Reject inference	The problem that outcomes are only known for applicants who were approved in the past.

APPENDIX B

ADDITIONAL TABLES AND FIGURES

TABLE X
FROM-SCRATCH NUMPY IMPLEMENTATIONS VS SCIKIT-LEARN (20,000 TRAINING ROWS, 10,000 CHECK ROWS).

Algorithm	Measure	Scratch	sklearn
Logistic reg. (Newton, MAP)	ROC-AUC	0.7544	0.7544
Bayesian logistic (Laplace)	ROC-AUC	0.7551	0.7544
Gaussian NB	ROC-AUC	0.6614	0.6614
PCA via SVD	var. in 30 comp.	65.7%	65.7%
Fisher LDA (30 comp.)	ROC-AUC	0.7531	0.7531
K-means++ ($k = 6$)	inertia	1,466,934	1,466,932
GMM with EM ($k = 4$)	avg log-lik.	−20.847	−20.851
MLP (32 hidden)	ROC-AUC	0.7053	0.7038

TABLE XI
FINAL LIGHTGBM HYPERPARAMETERS (OPTUNA TPE, TRIAL 23).

Parameter	Value
learning rate	0.0219
number of leaves	17
min. rows per leaf	426
row / feature sampling	0.951 / 0.272
L2 regularisation	0.457
trees (refit on all train rows)	1,871

TABLE XII
CALIBRATION ON VALIDATION (CROSS-FITTED).

	ROC-AUC	Brier	ECE
none (raw)	0.7931	0.06529	0.0024
Platt (sigmoid)	0.7930	0.06530	0.0017
isotonic	0.7908	0.06534	0.0012

TABLE XIII
CLASS-IMBALANCE EXPERIMENT (5-FOLD CV; ACTUAL DEFAULT RATE 0.081).

Variant	ROC-AUC	ECE	mean p
none	0.7870	0.004	0.079
class weights	0.7852	0.254	0.335
undersampling (1:2)	0.7831	0.189	0.270
imputed control	0.7858	0.004	0.079
SMOTE (1:2)	0.7831	0.003	0.084

TABLE XIV
HOW TO REPRODUCE (FROM THE REPOSITORY ROOT).

Command	What it does
<code>make data parquet</code>	download, convert to Parquet
<code>make splits features</code>	frozen splits, 240 features
<code>notebooks/01-14</code>	every number in this report
<code>make test</code>	133 unit tests
<code>make app / make api</code>	the app / the API
<code>make mlflow</code>	the experiment dashboard
<code>make report</code>	this PDF

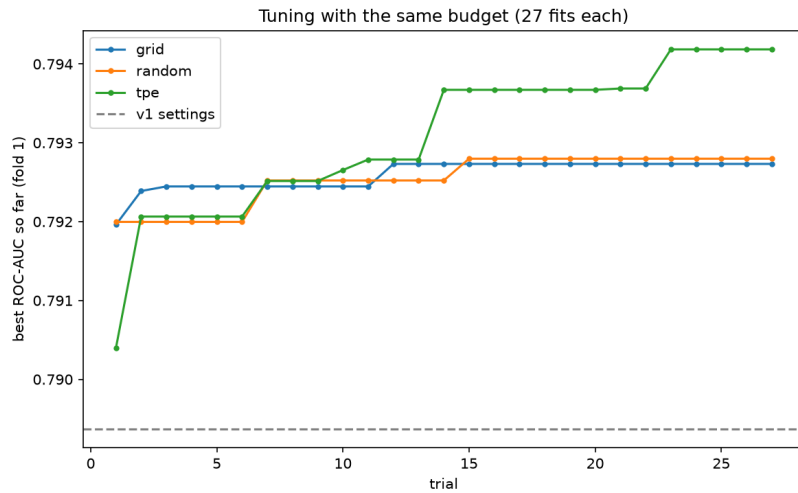


Fig. 11. Best ROC-AUC so far vs number of fits for grid, random and TPE search (fold 1).

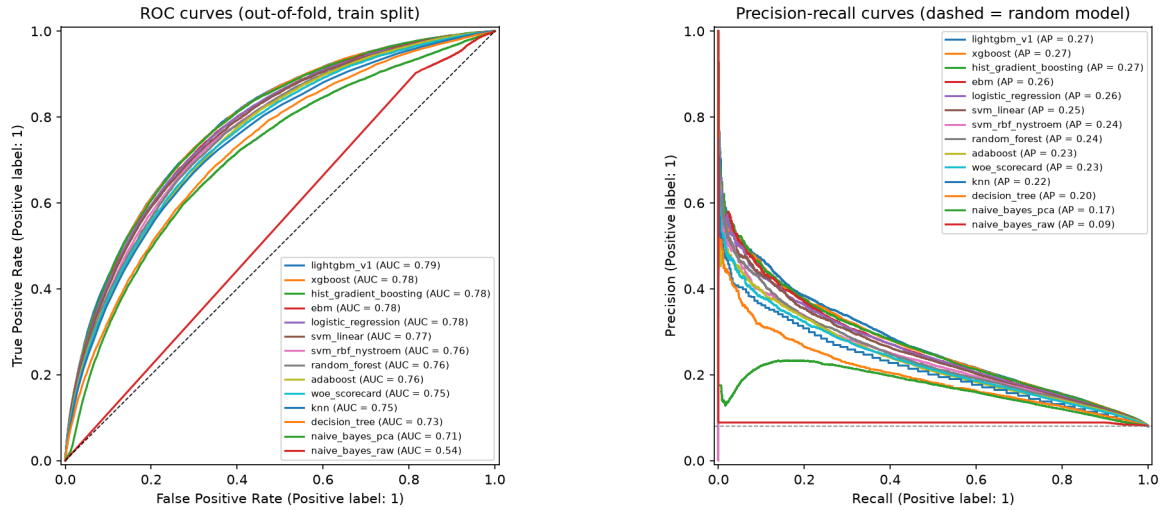


Fig. 12. ROC and precision-recall curves of the model ladder (out-of-fold).

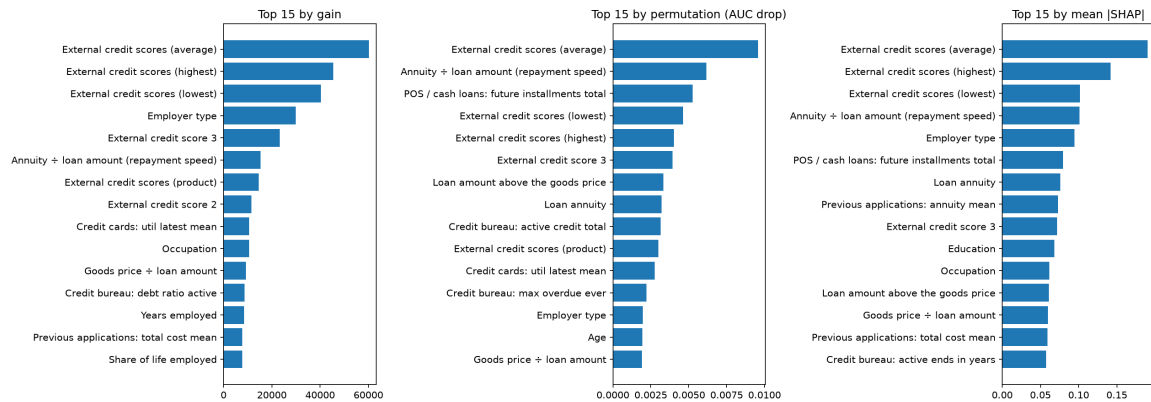


Fig. 13. Top features by gain, permutation and mean $|SHAP|$.

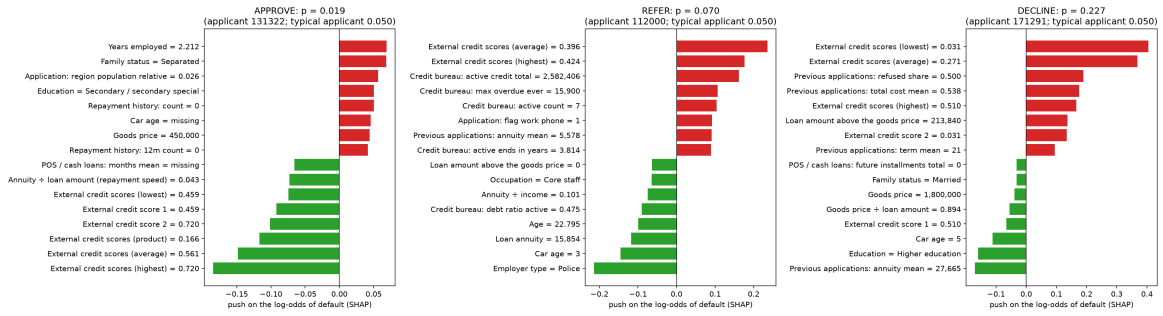


Fig. 14. Per-applicant SHAP reasons for an approved, a referred and a declined example.

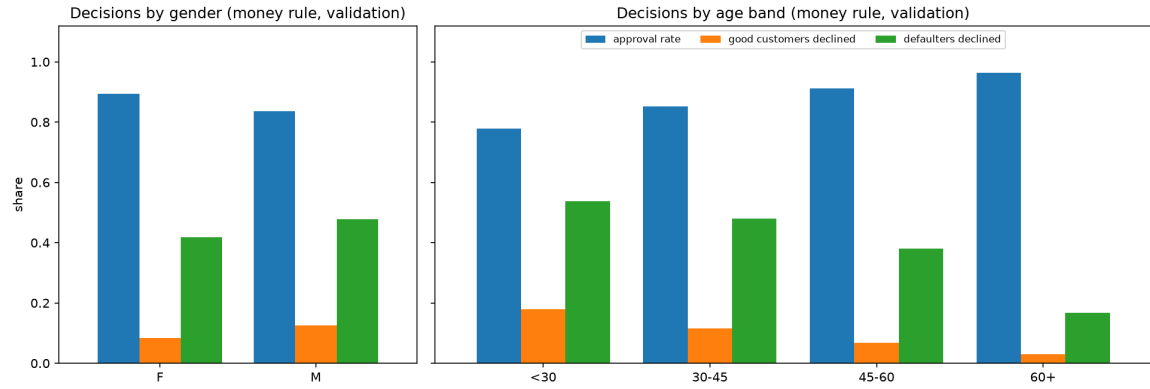


Fig. 15. Decision rates by gender and age band (money rule, validation).

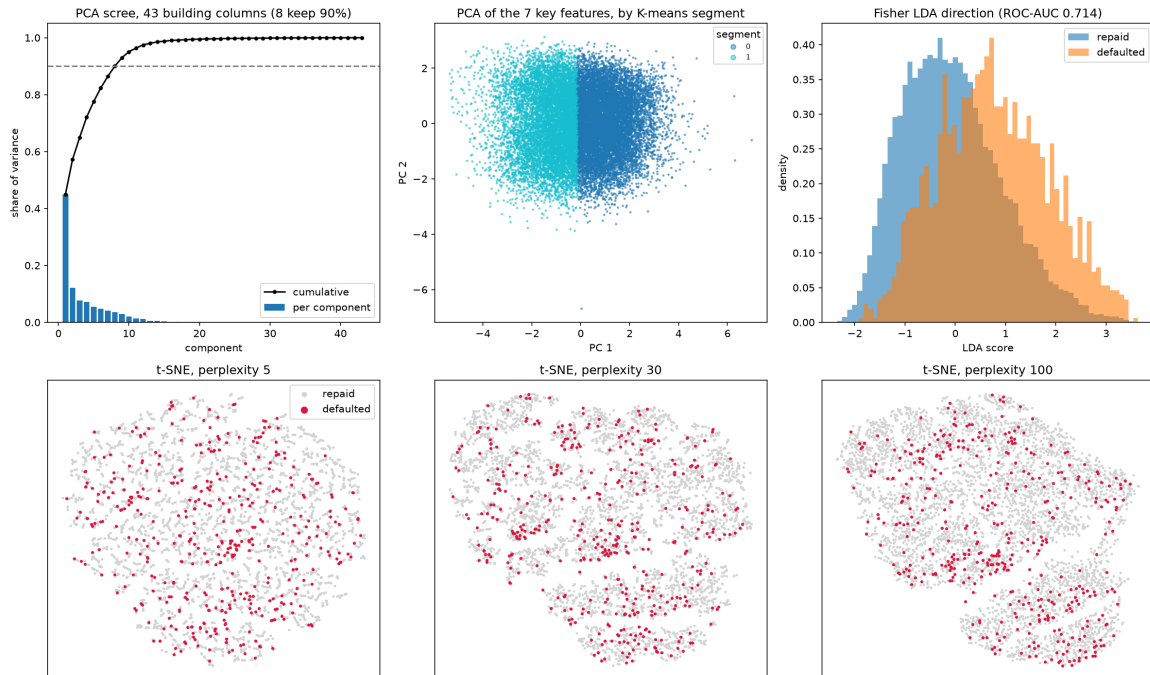


Fig. 16. PCA scree of the building columns, PCA of the key features, Fisher LDA projection, and t-SNE at three perplexities.